pontIA

APUNTES DE CLASE

Modelos de Generación de Imagen

5 de octubre de 2026

01 Casos de uso de generación de imagen en la industria

- **Sectores con casos de uso directos**
  Los casos de uso de generación de imagen están limitados a ciertas industrias: marketing, publicidad, moda, arte, retailing, e-commerce. En empresas estándar (bancos, consultoras, constructoras, energéticas) es difícil encontrar la necesidad de generar imágenes.
- **Generación de datos sintéticos**
  Es el caso de uso transversal a cualquier industria. Se generan imágenes sintéticas con modelos de difusión para ampliar un dataset cuando se tienen pocas muestras. Con ese dataset ampliado se entrena un modelo supervisado (clasificación de imagen, detección de fallos, etc.). El modelo final no es de difusión, sino de otra tipología (p. ej. clasificación).

> IDEA CLAVE
> **El caso de uso transversal a cualquier industria es la generación de datos sintéticos para ampliar datasets de entrenamiento.**

PontIA

02 Niveles de generación de imágenes con IA

- **Cuatro niveles de alto a bajo**
  Nivel 1: herramientas a nivel usuario (ChatGPT, Gemini). Nivel 2: generación a través de una API (Replicate). Nivel 3: uso de modelos open source en local (con notebook). Nivel 4: entrenar un modelo de difusión desde cero.
- **Mayor nivel, menor control**
  A mayor nivel, más sencillo de usar pero menor personalización y control sobre el proceso. A menor nivel, más complejidad técnica pero más control sobre el modelo y el resultado.

> IDEA CLAVE
> **Hay cuatro niveles de generación de imágenes con IA, ordenados de más alto nivel (usuario) a más bajo nivel (entrenar un modelo desde cero).**

03 Nivel 1: herramientas a nivel usuario

- **Uso sin código**
  Herramientas como ChatGPT o Gemini en el navegador permiten generar imágenes sin programar. Solo se necesita saber hacer un prompt. Cualquier persona con conocimientos básicos de usuario puede usarlas.

- **Personalización mínima**
  Lo único que se puede personalizar es el prompt. No hay control sobre qué modelo se ejecuta por detrás. Por ejemplo, OpenAI tiene varios modelos de generación de imagen y según la demanda o el momento puede usar un modelo más pequeño, más grande, más rápido o más lento. El control sobre el proceso es mínimo.

04 Nivel 2: consumo de modelos vía API

- **Concepto general**
  En lugar de usar un portal web, se llama directamente a la API de un proveedor (OpenAI, Google, Microsoft, etc.). El modelo está hosteado en la infraestructura del proveedor: disponibilidad, despliegue y actualizaciones de versión son transparentes para el usuario.
- **Coste e integración**
  La mayoría de APIs son de pago. Requiere conocimientos básicos de programación para hacer la llamada e integrarla en una aplicación.
- **Parámetros disponibles**
  Se elige el modelo concreto y se configuran parámetros como: tamaño de imagen de salida, calidad, prompt negativo (lo que no se quiere en la imagen), estilo, y en algunos casos desactivar el filtro de seguridad para contenido violento o sexual.
- **Uso en empresas**
  Este nivel y el siguiente (modelos open source en local) son los que más se usan en empresas para casos de uso reales.

> IDEA CLAVE
> **Las APIs de los proveedores permiten elegir modelo concreto y ajustar parámetros de generación, con infraestructura gestionada por el proveedor.**

05 Nivel 3: modelos open source en local

- **Cuándo se necesita**
  Cuando la regulación impide usar infraestructura externa (banca, defensa, legal) o cuando se quiere evitar el coste de las APIs de terceros.
- **Limitación de tamaño de modelo**
  Gestionar en local un modelo del mismo tamaño que los de los grandes proveedores es posible pero muy caro. En la práctica se trabaja con modelos medianos o pequeños, que no alcanzan el rendimiento de los modelos top.
- **Requisito de GPU**
  Los modelos de difusión necesitan GPU. Cada generación de imagen es un proceso de difusión inversa con múltiples pases por el modelo, lo que sin GPU resulta muy lento.
- **Gestión y equipo necesario**
  Toda la gestión de despliegue, disponibilidad y mantenimiento recae en el equipo propio. Se necesitan conocimientos de operaciones, producción y mantenimiento de servicios, no solo de consumo de modelos.

PontIA

- **Control total sobre la generación**
  Con el modelo en local se controlan todos los parámetros del proceso de difusión: número de pasos, peso del prompt de texto frente al ruido aleatorio, añadir capas, congelar capas o hacer fine-tuning del modelo.
- **Hugging Face para hosting local**
  Hugging Face permite hostear modelos open source en local de forma sencilla.

> IDEA CLAVE
> **Hostear modelos open source en local da control total sobre el proceso de generación, pero requiere GPU, equipo dedicado y asumir que se trabaja con modelos medianos o pequeños.**

06 Nivel 4: entrenar un modelo de difusión propio

- **Solo para investigación o didáctica**
  Entrenar un modelo de difusión desde cero da personalización máxima, pero hoy solo tiene sentido en laboratorios de investigación o con fines didácticos. En la práctica empresarial no compensa; lo que sí se hace es fine-tuning de modelos existentes.

07 Replicate: plataforma de modelos vía API

- **Qué es Replicate**
  Plataforma que actúa como repositorio de modelos generativos (imagen, vídeo, texto) y permite consumirlos todos a través de una sola API. Es una de las plataformas más populares para modelos de generación de imagen.
- **Interfaz común y abstracción**
  La API tiene una interfaz única: el código para llamar a cualquier modelo es el mismo. Se instala el paquete `replicate`, se llama a `replicate.run` con el model ID y el input (prompt y parámetros). Para cambiar de modelo basta con cambiar el nombre; no hay que tocar código. Esto es una ventaja frente a usar directamente la API de cada proveedor (la de OpenAI funciona diferente a la de Gemini, etc.).
- **Catálogo de modelos**
  Tiene modelos de todos los proveedores principales: Google, Recraft, Black Forest Labs (Flux), OpenAI, ByteDance, xAI, Leonardo AI, entre otros. Se actualiza constantemente.
- **Licencia comercial exclusiva**
  Algunos modelos, como Flux de Black Forest Labs, son open source y se pueden usar libremente, pero las imágenes generadas por la vía open source no tienen licencia comercial. Para usar las imágenes con fines comerciales hay que consumir el modelo a través de Replicate.
- **Playground de prueba**
  La interfaz web incluye un playground para generar imágenes de ejemplo y probar el comportamiento del modelo antes de pagar.
- **Diferencia con Freepik**
  Freepik (ahora fusionado con Magnific) es una herramienta de nivel 1 (se usa desde el navegador). Replicate es nivel 2: se consume por API desde código.

PontIA

- **Tarjeta del modelo**
  Al clicar un modelo en el catálogo de Replicate se abre su tarjeta. Lo primero que aparece es el Model ID, que es el identificador que hay que poner en la llamada a la API.
- **Parámetros de entrada**
  La tarjeta muestra los parámetros que admite la llamada. Los comunes son: `prompt` (obligatorio), `input_file` (imagen de referencia opcional), `aspect_ratio` (relación entre dimensiones de la imagen de salida), resolución y `output_format`. Algunos modelos añaden parámetros propios del proveedor; por ejemplo, un modelo de Google incluye un booleano para activar Google Web Search y generar imágenes basadas en información real.
- **Código Python y pricing**
  La tarjeta incluye un ejemplo en Python: se instala el paquete `replicate`, se configura un token de pago y se ejecuta un bloque de código para generar y guardar la imagen. Debajo aparece el pricing; por ejemplo, 14 imágenes por 1 dólar a resolución 1K o 66 imágenes por 10 dólares a 4K.
- **Playground**
  Replicate ofrece un playground donde, iniciando sesión, se puede escribir un prompt y generar imágenes para probar cómo se comporta cada modelo.

> IDEA CLAVE
> **Replicate ofrece una API con interfaz única para llamar a modelos de cualquier proveedor; cambiar de modelo es cambiar un nombre en el código.**

> IDEA CLAVE
> **La tarjeta del modelo en Replicate muestra el Model ID (necesario para la API), los parámetros de entrada, el pricing y un ejemplo de código en Python.**

PontIA

08 Elección de modelo y Artificial Analysis

- **Dataset de entrenamiento y capacidad actual**
  El conjunto de imágenes de entrenamiento influye en el estilo de los resultados (más realistas, más ilustración, etc.). Sin embargo, a día de hoy los modelos están tan avanzados que cualquiera resuelve correctamente la mayoría de tareas, así que esa diferencia queda más difuminada.
- **Ranking de modelos**
  Artificial Analysis es una página web que compara modelos de IA. En la sección Text to Image muestra un ranking con cada modelo y su proveedor. Aparecen modelos de OpenAI, Grok, Microsoft, Google (Nano Banana), Qwen, Seedream, etc. En el último mes se añadieron 12 modelos nuevos.
- **Image Arena: votación ciega**
  El ranking se construye con la puntuación que dan los usuarios en el Image Arena. Se presenta un prompt y dos imágenes generadas por modelos distintos sin revelar cuál es cuál. El usuario vota la mejor y esos votos alimentan la puntuación del ranking.
- **Comparativa de modelos**
  La página tiene un apartado de comparativa que cruza calidad frente a precio, tiempo de generación y coste de API. Se puede personalizar añadiendo los modelos que se deseen y comparar uno contra uno. También permite comparar proveedores directamente.

- **Artículos y otras categorías**
  Incluye un apartado de artículos con los reportes técnicos y explicativos que acompañan a cada release de modelo. Además de imagen, cubre generación de vídeo, música, agentes de código y chatbots.

> IDEA CLAVE
> **Artificial Analysis permite comparar modelos de generación de imagen por calidad, precio y velocidad mediante un ranking basado en votación ciega de la comunidad.**

09 Otras herramientas de generación de imagen

- **Alternativas de pago**
  Existen otras APIs y herramientas para generar imágenes con IA. Se menciona especialmente una enfocada a vídeo, que funciona como un editor muy completo: background removal, edición de imagen, pincel para seleccionar y modificar solo una zona concreta de la imagen, y generación de vídeo. Es más cara, pero muy completa.

10 Hugging Face: repositorio open source

- **Qué es Hugging Face**
  Es un repositorio open source de modelos de inteligencia artificial y de datasets. Los usuarios suben de manera altruista y gratuita sus modelos ya entrenados (texto, imagen, vídeo) y los datasets de entrenamiento, con código totalmente libre.
- **Comunidad y usuarios**
  No solo participan usuarios individuales: también empresas, organizaciones y departamentos académicos de universidades tienen perfil y publican sus desarrollos. Es un punto de encuentro de la comunidad de machine learning en general.
- **Librería Diffusers**
  Para interactuar con los modelos de difusión de Hugging Face se usa la librería Diffusers, que es una de las librerías que proporciona la plataforma.
- **Cuenta y menús principales**
  Crear una cuenta es gratuito. Los menús principales de la plataforma son modelos y datasets, que dan acceso a sus respectivos catálogos.
- **Catálogo de modelos**
  Hugging Face tiene 3 millones de modelos open source descargables. Se pueden filtrar por tipo de tarea (text to image, image to image, etc.), tamaño del modelo (parámetros) y ordenar por trending, más recientes o más descargas. Filtrando por text to image quedan unos 100 000 modelos. Entre los proveedores destacados: Stability AI, Black Forest Labs (Flux, sin licencia comercial) y Qwen (empresa china).
- **Catálogo de datasets**
  1 millón de datasets públicos, también filtrables por tamaño y tipo. Se pueden descargar para entrenar o ajustar modelos.

PontIA

- **Spaces, Buckets y servicios**
  Spaces es un playground donde la gente sube proyectos pequeños y permite probar modelos directamente en la interfaz de Hugging Face. Buckets es un servicio nuevo: servidores en la nube para entrenar modelos. Hugging Face nació como repositorio de modelos y datasets; la oferta de servidores es reciente. Descargar modelos y datasets es gratuito.

> IDEA CLAVE
> **Hugging Face es un repositorio open source donde la comunidad sube gratuitamente modelos entrenados y datasets de IA.**

> IDEA CLAVE
> **3 millones de modelos y 1 millón de datasets open source, descargables gratis; cada modelo tiene su propia licencia.**

11 Entorno de trabajo: Google Colab y librerías

- **Librerías necesarias**
  `diffusers` (modelos de difusión desde Hugging Face), `transformers` (arquitecturas subyacentes), `torch` (PyTorch), `accelerate` (optimización) y `opencv` (utilidades de imagen en Python).
- **GPU en Colab**
  Se necesita GPU para acelerar la generación. Al ejecutar la celda de comprobación debe aparecer Tesla T4 con 15 GB de VRAM. Si no, hay que seleccionar el entorno con GPU. Los recursos de Colab son limitados, así que se trabaja con modelos pequeños.

12 Modelo elegido: Dreamlike Art Diffusion 1.0

- **Tarjeta del modelo**
  En Hugging Face, cada modelo tiene una tarjeta con descripción, método de entrenamiento, ejemplos de imágenes, bloque de código para usarlo con `diffusers`, licencia y descargas mensuales. En Files and Versions está el repositorio con los archivos del modelo.
- **Tamaño y formato de pesos**
  Los pesos están en formato `safetensors` y ocupan 2 GB. Es un modelo muy pequeño, elegido por fines didácticos para caber en Colab. En un caso real se usarían modelos más grandes para obtener mejores resultados.

13 Pipeline de generación de imagen

- **No es una inferencia clásica**
  En machine learning clásico, la inferencia es pasar un input a un modelo y obtener un output. En generación de imagen es un pipeline completo con varios componentes.
- **Encoder de texto**
  El primer componente convierte el texto plano (prompt) en un embedding numérico (vector) que la UNet pueda comprender. Representa fielmente la información del texto en un vector numérico.

PontIA

- **Espacio latente y ruido aleatorio**
  Se parte de ruido aleatorio como input (la librería lo genera automáticamente). Ese ruido se codifica al espacio latente, un espacio de dimensiones reducidas respecto al espacio de la imagen (que puede ser 1024x1024). Trabajar en el espacio latente hace la predicción más rápida. Esta es la diferencia entre modelos de difusión puros y modelos de difusión latentes.

- **UNet: el modelo como tal**
  La UNet es la red neuronal entrenada. Recibe la imagen con ruido en el espacio latente y predice cuánto ruido hay. El ruido predicho se resta iterativamente hasta obtener la imagen final. La UNet no funciona sola: necesita el resto de componentes del pipeline.

> IDEA CLAVE
> **Generar una imagen no es una inferencia simple: es un pipeline con encoder de texto, espacio latente, UNet y proceso iterativo de eliminación de ruido.**

14 Repaso del entrenamiento de difusión

- **Diferencia entrenamiento vs inferencia**
  El proceso de entrenamiento y el de inferencia son distintos. En este ejercicio se usa un modelo ya entrenado (inferencia); el entrenamiento se aborda en ejercicios posteriores.

- **Cómo se entrena**
  Se coge una imagen original, se le añade ruido (sabiendo cuánto se añade) y se entrena la red neuronal para que, dada la imagen con ruido, prediga cuánto ruido contiene. Así el modelo aprende a restar el ruido y recuperar la imagen original.

- **Papel del texto en la eliminación de ruido**
  El texto condiciona la dirección en que se elimina el ruido. Aunque el ruido inicial es completamente aleatorio, el texto guía al modelo para generar la imagen correspondiente al prompt, no una imagen arbitraria. En el entrenamiento se incorpora el texto: dado una imagen con ruido y un texto, el modelo aprende qué ruido hay.

- **Inferencia iterativa de la UNet**
  En inferencia, la UNet ya entrenada recibe ruido codificado en espacio latente junto con el texto. En cada paso predice cuánto ruido hay, se resta ese ruido y se vuelve a pasar el resultado a la UNet. Así la imagen se va clarificando hasta que, tras todos los pasos (50, 100…), se obtiene un vector sin ruido que al decodificarlo produce la imagen final.

- **Dirección del denoising**
  La UNet quita ruido en la dirección que aprendió durante el entrenamiento. Si solo se entrenó con gatos, solo genera formas gatunas. Los modelos reales se entrenan con muchísimas imágenes de todo tipo y con descripciones de texto, por lo que el ruido se quita también en la dirección que indica el prompt.

> IDEA CLAVE
> **La generación de imagen no es una sola inferencia: es un proceso iterativo donde la UNet predice y resta ruido paso a paso, guiada por el texto.**

PontIA

15 Pipelines en Diffusers

- **Concepto de pipeline**
  En la librería Diffusers de Hugging Face, la clase principal para generar imágenes se llama Pipeline. Representa una tubería de múltiples componentes (encoder de texto, UNet, decoder…) que trabajan juntos.
- **AutoPipeline para text-to-image**
  `AutoPipelineForText2Image` detecta automáticamente qué pipeline usar según el modelo. Cada proveedor puede incluir componentes distintos: un encoder de texto diferente, capas adicionales para mejorar resolución o un safety checker que bloquea contenido violento o sexual.
- **Carga del modelo preentrenado**
  Se carga con `from_pretrained` indicando el Model ID del modelo en Hugging Face. Se puede especificar `use_safetensors=True` (formato de guardado de pesos) y la precisión: `float32` ocupa unos 4 GB y `float16` ocupa unos 2 GB. La diferencia de calidad entre ambas precisiones es imperceptible.

> IDEA CLAVE
> **La generación de una imagen no es inferencia sobre un solo modelo, sino una tubería (pipeline) de múltiples componentes donde uno de ellos es la inferencia sobre la UNet.**

PontIA

16 Componentes del pipeline de difusión

- **UNet**
  Red neuronal entrenada para predecir cuánto ruido hay en una imagen. Es el componente central del pipeline.
- **Text encoder y tokenizador**
  El text encoder (en este caso CLIP) convierte texto plano en embeddings (vectores numéricos). El tokenizador es una capa previa que convierte los tokens a índices numéricos antes de pasarlos al encoder.
- **VAE (Variational Autoencoder)**
  Agrupa el encoder y el decoder. El encoder comprime la imagen al espacio latente y el decoder la reconstruye.
- **Otros componentes**
  El feature extractor e image encoder pueden estar a nulo; algunos modelos usan un image encoder separado en lugar del autoencoder conjunto. El safety checker es una capa de seguridad opcional que bloquea generaciones de contenido inapropiado. El scheduler acompaña a la UNet e indica cuántos pasos de difusión hacia atrás se realizan.

17 Generación de imágenes y pasos de difusión

- **Llamada básica al pipeline**
  Se llama al objeto pipeline con el parámetro `prompt` (obligatorio) y `num_inference_steps` (número de pasos de difusión hacia atrás, por defecto 50). Cada paso es una inferencia a la UNet.

- **Efecto del número de pasos**
  Con 2 pasos la imagen sale prácticamente como ruido. Con 50 pasos ya se obtiene una imagen reconocible. Como regla general, entre 100 y 150 pasos la calidad se estabiliza y más pasos no la mejoran. Menos pasos acelera la generación pero reduce la calidad.
- **Control local vs API**
  Al ejecutar el modelo en local se tiene control total de estos parámetros. Cuando se genera a través de ChatGPT o una API externa, no se pueden ajustar.

> IDEA CLAVE
> **A partir de ~100-150 pasos la calidad ya no mejora; menos pasos implica más velocidad pero menor calidad.**

18 Aleatoriedad y semilla

- **Origen de la aleatoriedad**
  Todos los componentes del pipeline (UNet, encoder, CLIP) están congelados tras el entrenamiento. El único componente aleatorio es el ruido de entrada. Sin semilla, cada ejecución genera un ruido distinto y por tanto una imagen distinta.
- **Fijar la semilla**
  Se puede fijar una semilla con el parámetro `generator` del pipeline. Así el ruido de entrada es siempre el mismo y, con el mismo prompt, se reproduce la misma imagen.

PontIA

19 Parámetros avanzados del pipeline

- **Guidance scale**
  Controla la importancia relativa del texto frente al vector latente al combinarlos. Normalmente se usa una suma ponderada y el `guidance_scale` es el factor de ponderación del texto.
- **Otros parámetros configurables**
  Se puede ajustar `height` y `width` de la imagen, `negative_prompt` (lo que no debe aparecer), número de imágenes por prompt, `eta` y `sigmas` (factores de la ecuación de difusión). Con `prompt_embeds` se pasan directamente embeddings de texto ya calculados, saltándose el text encoder. Con `latents` se pasan vectores latentes directamente a la UNet, saltándose el encoder.
- **Control total frente a servicios cerrados**
  Con el pipeline se puede ajustar tamaño, número de pasos, peso del texto (`guidance_scale`), reutilizar prompts de otro modelo, etc. En ChatGPT o Gemini solo se lanza el prompt sin control sobre el proceso. Es más complejo, pero la posibilidad está ahí.
- **Calidad del modelo pequeño**
  El modelo descargado genera imágenes razonables para su tamaño: no comete errores graves, aunque hoy existen modelos bastante mejores.

> IDEA CLAVE
> **Usar el pipeline con Diffusers da un control del 100 % sobre la generación, frente a lanzar un prompt en ChatGPT o Gemini y cruzar los dedos.**

20 ControlNet e Image to Image

- **ControlNet condicionada a bordes**
  Se parte de una imagen de referencia, se extraen sus bordes y se genera una imagen nueva que respeta el contorno pero aplica un prompt distinto. Recurso dejado para práctica autónoma.
- **Image to Image**
  Dada una imagen de referencia, se mantiene el estilo general pero se modifica el contenido según un nuevo prompt. También dejado como recurso para práctica autónoma.

21 Generación con semilla fija

- **Eliminar la aleatoriedad**
  El generador por defecto no tiene semilla. Se define un generador con `manual_seed` (cualquier número) y se pasa en el parámetro `generator` del pipeline. Con esto se elimina el componente aleatorio y la imagen generada es siempre la misma.

> IDEA CLAVE
> Para obtener siempre la misma imagen hay que crear un generador con `manual_seed` y pasarlo como parámetro `generator` al pipeline.

22 Tamaño de modelos de imagen vs. texto

- **Rangos similares de parámetros**
  Tanto en imagen como en texto hay modelos pequeños y gigantescos (200 GB, 500 GB o más). La arquitectura difiere: los LLM usan un Transformer y los modelos de difusión suelen usar una UNet, pero el número de parámetros en gamas altas es similar.
- **Modelos pequeños de imagen vs. texto**
  Un modelo de texto pequeño difícilmente da resultados decentes; en cambio, un modelo de imagen pequeño puede producir imágenes aceptables, como se ha visto en el ejercicio.

23 Entrenamiento de un modelo de difusión desde cero

- **Objetivo del ejercicio**
  Se abre un segundo notebook para crear y entrenar un modelo de difusión desde cero (cuarto nivel de profundidad). Nadie entrena uno desde cero en producción, pero el ejercicio ayuda a entender el proceso de entrenamiento y sirve de base para los entrenamientos del día siguiente.
- **Solo PyTorch**
  Se usa únicamente PyTorch (y Matplotlib para visualización), sin librerías de alto nivel, para ver el proceso desde lo más básico. Se necesita GPU, por lo que hay que cerrar sesiones anteriores en Colab.

PontIA

- **Componente a entrenar: la UNet**
  La UNet es el componente que realiza el proceso de difusión hacia atrás (eliminar ruido). El text encoder (como CLIP) y el VAE también son entrenables, pero se reutilizan preentrenados porque su comportamiento ya es bueno. Para que el modelo genere imágenes concretas, es la UNet la que hay que reentrenar.

- **Planteamiento supervisado**
  Es un entrenamiento supervisado. La variable explicativa (X) es una imagen con ruido. La variable objetivo (Y) es el ruido presente en esa imagen, es decir, la UNet aprende a predecir qué ruido contiene la imagen.

- **Construcción del dataset de entrenamiento**
  La variable de entrada (X) es una imagen a la que se le ha añadido ruido. La variable objetivo (Y) es exactamente el ruido añadido. Como el ruido lo introduce uno mismo, se conoce la variable objetivo sin necesidad de etiquetado manual. Es el mismo paralelismo que un modelo de regresión clásico: la predicción se compara con la Y real, se obtiene un error y se optimiza el modelo.

- **Ventaja de los modelos de difusión**
  El dataset de entrenamiento no necesita estar etiquetado. Basta con descargar imágenes de internet y añadirles ruido. Si se quiere incorporar texto, normalmente se usa el pie de foto que acompaña a la imagen en internet como texto descriptivo.

- **Dataset Fashion MNIST**
  El ejercicio práctico usa el dataset Fashion MNIST: imágenes de 16x16 píxeles en blanco y negro (1 canal) de objetos de moda (botas, camisetas, pantalones, bolsos). Se definen como constantes el tamaño de imagen, el número de canales y el batch size de entrenamiento.

- **Proceso de difusión hacia adelante y timesteps**
  El proceso de difusión hacia adelante añade ruido a las imágenes. Depende de una variable T (timesteps) que controla la cantidad de ruido. A cada imagen del dataset se le asigna un nivel de ruido aleatorio (no todos los niveles a cada imagen). Como se entrena a gran escala con millones de imágenes, al final se cubren todos los niveles. El modelo debe aprender a quitar ruido en cualquier situación, desde poco ruido hasta mucho.

- **Scheduler de ruido (beta)**
  Se definen 150 pasos de ruido. El scheduler lineal va de beta inicial 0,001 a beta final 0,02. También existe el scheduler de coseno, que añade el ruido de forma más gradual: evita meter demasiado ruido en etapas tempranas y concentra más ruido al final. En el ejercicio se usa el lineal por simplicidad.

- **Fórmula para añadir ruido**
  Imagen con ruido = imagen original (x0) multiplicada por un factor + ruido de una distribución normal multiplicado por otro factor. En etapas tempranas el factor de ruido es pequeño; en etapas tardías es grande. La función Q (difusión hacia adelante) recibe x0 y un valor t (entre 0 y 150), y devuelve la imagen con ruido (xt) y el ruido añadido, generado con `torch.randn`.

- **Arquitectura de la UNet**
  La UNet tiene forma de U. Los bloques down reducen la dimensión de la imagen mediante capas de convolución (con ReLU, normalización y max pooling). Los bloques up incrementan la dimensión mediante capas de convolución transpuesta. Entre ambos hay skip connections que pasan información de los bloques down a los up. También incluye capas de atención.

PontIA

- **Forward de la UNet**
  La entrada de la UNet es la imagen con ruido (xt) y el timestep (t). La imagen pasa por los bloques down hasta llegar a un vector latente. Ahí se inyecta la información del timestep t para indicar a la red cuánto ruido hay (por ejemplo, t=5 implica poco ruido, t=150 mucho). Después se ejecutan los bloques up para volver a las dimensiones originales.

- **Función de coste**
  La función de coste mide el error de predicción y es la que se minimiza durante el entrenamiento. La red debe predecir la cantidad de ruido presente en la imagen; la diferencia entre el ruido predicho y el real es el error que se optimiza.

- **Función de pérdida**
  Se usa el mean squared error (MSE) entre el ruido real (el que se ha introducido) y el ruido predicho por la red. Si la predicción es perfecta, el error es cero. Un error grande indica que la red debe actualizar sus parámetros.

- **Bucle de entrenamiento**
  Se define un optimizador Adam con un learning rate y un número de épocas. Se activa `model.train()` para que los parámetros sean modificables. Para cada época se recorre el dataset imagen a imagen (en batches). Para cada imagen: 1) se elige un t aleatorio entre 0 y T con `torch.randint`, 2) se aplica la función q para obtener la imagen con ruido y el ruido añadido, 3) se pasa la imagen con ruido y t a la UNet, que devuelve una predicción del ruido, 4) se calcula el loss con `F.mse_loss` entre ruido real y predicho, 5) se hace `loss.backward()` (propagación hacia atrás) y `optimizer.step()` (actualización de parámetros).

- **El loss como objeto de PyTorch**
  El loss que devuelve `F.mse_loss` no es un simple valor numérico: es un objeto de PyTorch vinculado al grafo computacional. Por eso puede llamar a `loss.backward()` y relacionarse con el optimizador y los parámetros del modelo para hacer la propagación hacia atrás y la actualización.

- **Estructura idéntica a ML clásico**
  La estructura del bucle es la misma que en cualquier entrenamiento de redes neuronales supervisado. Las variables de entrada son la imagen con ruido y el time step t; la variable objetivo es el ruido. Se compara la predicción con el valor real, se obtiene el error y se propaga hacia atrás. El 90 y pico por ciento de los modelos de generación de imagen entrenan la UNet de esta manera; las variaciones están en qué text encoder, autoencoder o capas de refinamiento se usan.

- **Gran ventaja del enfoque**
  Se pueden usar todas las imágenes de internet como dataset sin necesidad de que estén etiquetadas de ninguna manera.

> IDEA CLAVE
> **La UNet es el componente que se entrena; el text encoder y el VAE se reutilizan preentrenados.**

> IDEA CLAVE
> **El dataset se construye sin etiquetado manual: a cada imagen original se le añade un ruido controlado que se convierte en la variable objetivo.**

> IDEA CLAVE
> **Es un problema de ML supervisado: la entrada es la imagen con ruido y el time step t, la variable objetivo es el ruido real, y la red aprende a predecirlo.**

PontIA

24 Train, test y evaluación en modelos de difusión

- **Se aplica la misma separación**
  En un desarrollo a gran escala (p. ej. Stable Diffusion) hay que separar train, evaluación y test. El conjunto de evaluación sirve para saber cuándo detener el entrenamiento (cuando el loss deja de mejorar). El de test sirve para comprobar que no hay overfitting (que el loss en test sea parecido al de entrenamiento). En el ejercicio del notebook no se hace porque es un ejemplo simplificado.
- **Métricas menos claras**
  A diferencia de otros problemas de ML, aquí no hay métricas como precision o accuracy. La única métrica cuantitativa es el loss. La validación real acaba siendo más subjetiva: si las imágenes generadas tienen buena pinta, el modelo funciona.
- **Variable objetivo generada por uno mismo**
  Que la variable objetivo (el ruido) la genere uno mismo no afecta a la validez del esquema train/test. El test sigue sirviendo para evaluar si el entrenamiento ha ido bien: se reservan imágenes no vistas, se les mete ruido, se evalúa la predicción y, si es mala, el entrenamiento ha fallado. Es una forma universal de comprobar que un entrenamiento ha funcionado.

> IDEA CLAVE
> **La separación train/test/evaluación se aplica igual que en cualquier problema de ML, pero la única métrica cuantitativa disponible es el loss.**

PontIA

25 Inferencia del modelo entrenado

- **Arquitectura sin texto ni espacio latente**
  En este ejercicio no hay text encoder ni variational autoencoder. Se trabaja directamente con la UNet sobre el tamaño de la imagen original (canales, 16×16), sin espacio latente.
- **Proceso de generación**
  Se genera un vector de ruido aleatorio con `torch.randn` del tamaño de la imagen. Iterativamente desde T hasta 0: se pasa la imagen con ruido y el t actual al modelo, se obtiene el ruido predicho y se resta de la imagen con ruido. Cada iteración produce una imagen con menos ruido que se vuelve a pasar al modelo.
- **Resultados**
  En 10 pasos de generación (se muestran 5) se obtienen imágenes reconocibles: una bota claramente identificable, una camiseta clara y otras formas más difusas. El modelo ha aprendido la idea general aunque los resultados no son perfectos por la simplicidad del ejercicio.